PEP 604 – Allow writing union types as `X | Y`

Author: Philippe PRADOS <python at prados.fr>, Maggie Moss <maggiebmoss at gmail.com>
Sponsor: Chris Angelico <rosuav at gmail.com>
BDFL-Delegate: Guido van Rossum <guido at python.org>
Discussions-To: Typing-SIG list (https://mail.python.org/archives/list/typing-sig@python.org/)
Status: Final
Type: Standards Track
Topic: Typing (../topic/typing/)
Created: 28-Aug-2019
Python-Version: 3.10
Post-History: 28-Aug-2019, 05-Aug-2020

Source: https://github.com/python/peps/blob/main/peps/pep-0604.rst

Last modified: 2024-02-16 17:06:07 UTC

Important ×

This PEP is a historical document. The up-to-date, canonical documentation can now be found at Union Type (https://docs.python.org/3/library/stdtypes.html#types-union).

See PEP 1 (../pep-0001/) for how to propose changes.

Abstract (#abstract)

This PEP proposes overloading the `|` operator on types to allow writing `Union[X, Y]` as `X | Y`, and allows it to appear in `isinstance` and `issubclass` calls.

Motivation (#motivation)

PEP 484 (../pep-0484/) and PEP 526 (../pep-0526/) propose a generic syntax to add typing to variables, parameters and function returns. PEP 585 (../pep-0585/) proposes to expose parameters to generics at runtime (../pep-0585/#parameters-to-generics-are-available-at-runtime). Mypy [1] (#id5) accepts a syntax which looks like:

```
annotation: name_type
name_type: NAME (args)?
args: '[' paramslist ']'
paramslist: annotation (',' annotation)* [',']
```

- To describe a disjunction (union type), the user must use `Union[X, Y]`.

The verbosity of this syntax does not help with type adoption.

Proposal (#proposal)

Inspired by Scala [2] (#id6) and Pike [3] (#id7), this proposal adds operator `type.__or__()`. With this new operator, it is possible to write `int | str` instead of `Union[int, str]`. In addition to annotations, the result of this expression would then be valid in `isinstance()` and `issubclass()`:

```
isinstance(5, int | str)
issubclass(bool, int | float)
```

We will also be able to write `t | None` or `None | t` instead of `Optional[t]`:

```
isinstance(None, int | None)
isinstance(42, None | int)
```

Specification (#specification)

The new union syntax should be accepted for function, variable and parameter annotations.

Simplified Syntax (#simplified-syntax)

```
# Instead of
# def f(list: List[Union[int, str]], param: Optional[int]) -> Union[float, str]
def f(list: List[int | str], param: int | None) -> float | str:
    pass

f([1, "abc"], None)

# Instead of typing.List[typing.Union[str, int]]
typing.List[str | int]
list[str | int]

# Instead of typing.Dict[str, typing.Union[int, float]]
typing.Dict[str, int | float]
dict[str, int | float]
```

The existing `typing.Union` and `|` syntax should be equivalent.

```
int | str == typing.Union[int, str]

typing.Union[int, int] == int
int | int == int
```

The order of the items in the Union should not matter for equality.

```
(int | str) == (str | int)
(int | str | float) == typing.Union[str, float, int]
```

Optional values should be equivalent to the new union syntax

```
None | t == typing.Optional[t]
```

A new Union.__repr__() method should be implemented.

```
str(int | list[str])
# int | list[str]

str(int | int)
# int
```

isinstance and issubclass (#isinstance-and-issubclass)

The new syntax should be accepted for calls to `isinstance` and `issubclass` as long as the Union items are valid arguments to `isinstance` and `issubclass` themselves.

```
# valid
isinstance("", int | str)

# invalid
isinstance(2, list[int]) # TypeError: isinstance() argument 2 cannot be a parameterized generic
isinstance(1, int | list[int])

# valid
issubclass(bool, int | float)

# invalid
issubclass(bool, bool | list[int])
```

Incompatible changes (#incompatible-changes)

In some situations, some exceptions will not be raised as expected.

If a metaclass implements the `__or__` operator, it will override this:

```
>>> class M(type):
...     def __or__(self, other): return "Hello"
...
>>> class C(metaclass=M): pass
...
>>> C | int
'Hello'
>>> int | C
typing.Union[int, __main__.C]
>>> Union[C, int]
typing.Union[__main__.C, int]
```

Objections and responses (#objections-and-responses)

For more details about discussions, see links below:

- Discussion in python-ideas (https://mail.python.org/archives/list/python-ideas@python.org/thread/FCTXGDT2NNKRJQ6CDEPWUXHVG2AAQZZY/)
- Discussion in typing-sig (https://mail.python.org/archives/list/typing-sig@python.org/thread/D5HCB4NT4S3WSK33WI26WZSFEXCEMNHN/)

1. Add a new operator for `Union[type1, type2]`? (#add-a-new-operator-for-union-type1-type2)

PROS:

- This syntax can be more readable, and is similar to other languages (Scala, …)
- At runtime, `int|str` might return a simple object in 3.10, rather than everything that you'd need to grab from importing `typing`

CONS:

- Adding this operator introduces a dependency between `typing` and `builtins`
- Breaks the backport (in that `typing` can easily be backported but core `types` can't)
- If Python itself doesn't have to be changed, we'd still need to implement it in mypy, Pyre, PyCharm, Pytype, and who knows what else (it's a minor change see "Reference Implementation")

2. Change only PEP 484 (Type hints) to accept the syntax `type1 | type2` ? (#change-only-pep-484-type-hints-

to-accept-the-syntax-type1-type2)

PEP 563 (../pep-0563/) (Postponed Evaluation of Annotations) is enough to accept this proposition, if we accept to not be compatible with the dynamic evaluation of annotations (`eval()`).

```
>>> from __future__ import annotations
>>> def foo() -> int | str: pass
...
>>> eval(foo.__annotations__['return'])
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<string>", line 1, in <module>
TypeError: unsupported operand type(s) for |: 'type' and 'type'
```

3. Extend `isinstance()` and `issubclass()` to accept `Union` ? (#extend-isinstance-and-issubclass-to-accept-union)

```
isinstance(x, str | int) ==> "is x an instance of str or int"
```

PROS:

- If they were permitted, then instance checking could use an extremely clean-looking notation

CONS:

- Must migrate all of the `typing` module in `builtin`

Reference Implementation (#reference-implementation)

A new built-in `Union` type must be implemented to hold the return value of `t1 | t2`, and it must be supported by `isinstance()` and `issubclass()`. This type can be placed in the `types` module. Interoperability between `types.Union` and `typing.Union` must be provided.

Once the Python language is extended, mypy [1] (#id5) and other type checkers will need to be updated to accept this new syntax.

- A proposed implementation for cpython is here (https://github.com/python/cpython/pull/21515).
- A proposed implementation for mypy is here (https://github.com/pprados/mypy/tree/PEP604).

References (#references)

[1] *(1 (#id1), 2 (#id4))*
mypy http://mypy-lang.org/ (http://mypy-lang.org/)

[2 (#id2)]
Scala Union Types https://dotty.epfl.ch/docs/reference/new-types/union-types.html
(https://dotty.epfl.ch/docs/reference/new-types/union-types.html)

[3 (#id3)]
Pike http://pike.lysator.liu.se/docs/man/chapter_3.html#3.5 (http://pike.lysator.liu.se/docs/man/chapter_3.html#3.5)

Copyright (#copyright)

This document is placed in the public domain or under the CC0-1.0-Universal license, whichever is more permissive.